ALD-SC
A Spectral Latent Diffusion Model

Arrowspace Latent Diffusion with Spectral Chart Conditioning

Lorenzo Moriondo
tuned.org.uk
ORCID: 0000-0002-8804-2963
tunedconsulting@gmail.com

July 2026

tuned-org-uk/arrowspace-latent-diffusion
tuned-org-uk/entropic-semantic-diffusion

Abstract

We present ALD-SC (*Arrowspace Latent Diffusion with Spectral Chart Conditioning*), a spectral latent diffusion model in which decoding is performed on the feature-space manifold defined by a frozen ArrowSpace graph Laplacian L_F and its associated energy-dispersion network λ^{ED}.

The model encodes each image into a 2.5-D representation: a spatial VAE latent z for local detail, and a feature field A projected onto the smooth eigenspace of L_F. The 2.5-D encoding target is the DualSpaceMatrix $M_N = \alpha\|VV^\top\|_F - \beta\|VL_FV^\top\|_F$, which fuses item-space geometry with feature-space topology. From the projected feature field we extract a compact spectral chart $c_{\mathrm{spec}} \in \mathbb{R}^{3q}$ comprising normalised band energies, the projected dispersion prior, and the Laplacian eigenvalues.

Diffusion operates only on z with v-prediction and a cosine schedule; c_{spec} is a conditioning side-channel, not a second diffused state. The DiT denoiser receives c_{spec} via adaptive layer normalisation, with classifier-free guidance dropout.

The decoder is the research contribution: a *graph-structured* decoder in which L_F (via its eigenvectors U_q) defines the reconstruction paths and λ^{ED} (via c_{spec}) gates the energy allocation at each resolution. The *WaveReconstructionBlock* projects feature activations onto the chart basis, gates them by the dispersion-derived weights, and lifts them back — the graph-theoretic analogue of the VAE reparameterization trick, replacing unconstrained convolutions with propagation along the graph's smooth directions.

The Barontini entropic clock is an active part of both inference and decoding. The per-mode schedule $\tau_k(t) = \nu_k \cdot t$ and the heat-death criterion $\sum_k \nu_k \bar{\alpha}_k(t) < \varepsilon$ terminate sampling intrinsically rather than at a fixed step count. The *ClockGatedGraphDecoder* modulates decoding tempo by $\bar{\alpha}_k(t)$: graph-structured reconstruction effort increases as the denoising process resolves each spectral mode.

The full pipeline — prior construction, spectral VAE, graph decoder, v-prediction diffusion training, DDIM/Euler sampling with spectral stopping, and end-to-end image generation — is implemented in PyTorch and validated by 107 unit tests.

The central falsifiable claim is that decoding on the feature-space manifold yields better global semantic coherence under compression than decoding on an unconstrained ambient latent.

Keywords: latent diffusion, ArrowSpace, graph Laplacian, spectral conditioning, feature-space geometry, v-prediction, classifier-free guidance, thermal time.

Contents

1 Introduction

Latent diffusion models achieve high-quality image synthesis by moving the iterative denoising process from pixel space to a compressed latent space [6]. While effective, this design leaves the semantic structure of the latent space largely unconstrained: smoothness, coherence, and global feature organisation emerge only indirectly from reconstruction and denoising losses.

The predecessor ESDM [8] proposed a spectrally conditioned diffusion architecture in which a frozen ArrowSpace graph Laplacian [4, 5] defines the valid semantic subspace, while a full entropic clock — wave recurrence, vibrational pump, and a bright–dark KL entropy clock inspired by Barontini's cold-atom experiment [2] — drives the diffusion schedule from internal thermodynamics. That model is theoretically rich but implementation-heavy.

This paper strips ESDM down to its testable core. We retain the frozen ArrowSpace prior and the 2.5-D latent (a spatial latent plus a spectral chart) but replace the entropic clock with a standard cosine schedule and v-prediction. The spectral chart enters as a *conditioning side-channel*, not as a second diffused state. The result is a model that is recognisably a latent diffusion system but whose semantic organisation is explicitly governed by graph Laplacian structure, trainable on a single CPU in minutes, and implementable in under 700 lines.

What is implemented. The code at `arrowspace-latent-diffusion` implements:

- The frozen ArrowSpace prior $(L_F, U_q, \Lambda_q, \lambda^{\mathrm{ED}})$ via the ArrowSpace library or a kNN fallback (`wire_graph.py`, `arrow_prior.py`, `build_prior.py`).
- The 2.5-D encoding target: the DualSpaceMatrix $M_N = \alpha\|VV^\top\|_F - \beta\|VL_FV^\top\|_F$ (`dual_space.py`).
- The spectral VAE with dual-head encoder producing (z, A) and $c_{\mathrm{spec}} \in \mathbb{R}^{3q}$ (`vae.py`, `losses.py`).
- The DiT denoiser with AdaLN conditioning on c_{spec} and classifier-free guidance dropout (`dit.py`).
- Cosine and linear noise schedules with v-prediction (`schedule.py`).
- A graph-structured decoder where L_F (via U_q) defines reconstruction paths and λ^{ED} (via c_{spec}) gates energy allocation. The `WaveReconstructionBlock` propagates information along the graph's smooth directions — the graph-theoretic analogue of the VAE reparameterization trick (`graph_decoder.py`).
- The Barontini entropic clock as an active intrinsic stopping criterion: the sampler terminates when $\sum_k \nu_k \bar{\alpha}_k(t) < \varepsilon$ (`spectral_schedule.py`, `sampling.py`). The `ClockGatedGraphDecoder` also modulates decoding tempo by $\bar{\alpha}_k(t)$.
- DDIM and Euler samplers, end-to-end image generation via CLI (`sampling.py`, `scripts/sample.py`).

The full entropic-clock formulation — wave recurrence, vibrational pump, density matrices, and the bright–dark KL clock — is implemented in the predecessor repository `entropic-semantic-diffusion` and is cited here as background. The code is validated by 107 unit tests on CPU.

The three commitments of ALD-SC are:

1. a frozen ArrowSpace prior $(L_F, U_q, \Lambda_q, \lambda^{\mathrm{ED}})$ built once from corpus feature statistics;

2. a 2.5-D latent consisting of a standard spatial latent plus a compact spectral chart c_{spec};

3. a decoder and denoiser conditioned by feature-space spectral geometry.

2 Background

2.1 ArrowSpace and feature-space geometry

ArrowSpace [4] introduces a graph-based treatment of vector spaces in which feature dimensions, rather than only data points, participate in a structural semantic geometry. Given a corpus of embeddings with feature dimension F, one constructs a feature-space graph and its Laplacian $L_F \in \mathbb{R}^{F\times F}$. The eigenvectors of L_F define smooth and rough directions in feature space, while the Energy Dispersion Networks formulation [5] treats the feature columns of an embedding matrix as nodes in a semantic energy network.

In ALD-SC we treat ArrowSpace as a frozen semantic prior:

$$\mathcal{G}_{\text{Arrow}} = (L_F, U_q, \Lambda_q, \lambda^{\text{ED}}),$$

where $U_q \in \mathbb{R}^{F\times q}$ contains the retained smooth modes, $\Lambda_q = \text{diag}(\nu_1, \ldots, \nu_q)$ are the corresponding eigenvalues, and $\lambda^{\text{ED}} \in [0,1]^F$ is an empirical energy-dispersion distribution over feature nodes. This prior is built once from the training corpus and never updated during training (all components are stored as non-trainable buffers).

2.2 Latent diffusion

Latent diffusion models [6] shift the forward and reverse diffusion processes from pixel space into a learned compressed latent representation. An image $x \in \mathbb{R}^{H\times W\times 3}$ is encoded into a latent tensor $z \in \mathbb{R}^{h\times w\times c}$, denoised over a sequence of timesteps, and decoded back to RGB. Standard latent diffusion assumes that the VAE latent is sufficient as the sole state on which the generative process must operate. Our claim is that this is lossy: a spatial latent is effective for local visual detail, but a separate feature-space spectral state is needed to preserve global semantic organisation under compression [5].

2.3 Internal time and entropic motivation

The original ESDM framing was motivated by the problem of time in closed systems. In [3] it is argued that in a generally covariant theory, the physical time-flow may arise from the thermodynamical state of the system rather than from a universal external parameter. Barontini's cold-atom experiment [2] provides an operational testbed: a closed system partitioned into observed and unobserved sectors admits an entropic time constructed from internal entropy exchange. We retain this as a design principle — generative dynamics should be informed by state-derived geometry — but in ALD-SC it is instantiated through a frozen spectral prior rather than through a fully entropic recurrence. The full entropic clock is deferred to Section 8 as a theoretical extension.

3 Design transition: from ESDM to ALD-SC

Earlier ESDM drafts coupled five ingredients tightly: a dual-space semantic operator (ArrowSpace), a wave recurrence over item states, a bright–dark sector entropy clock, an entropic pump, and a topology decoder acting as an inverse spectral map. This unified picture was conceptually rich but implementation-heavy.

Remark 1. *The strongest reusable result of* ESDM *is not the full second-order entropic dynamics but the geometric constraint that generation should be restricted to a smooth ArrowSpace subspace and decoded relative to a feature-space manifold.*

ALD-SC therefore keeps:

- the frozen ArrowSpace prior $(L_F, U_q, \Lambda_q, \lambda^{\mathrm{ED}})$;
- the projection onto smooth spectral modes $\Pi_q = U_q U_q^\top$;
- the 2.5-D representation (spatial latent + spectral chart);
- the heat-kernel interpretation of the spectral chart;
- topology-adaptive decoding via spectral gates.

To provide an end-to-end minimal implementation, it **removes** from the predecessor ESDM codebase:

- the full wave recurrence;
- explicit Rayleigh-gradient restoring forces;
- the bright–dark KL clock as the primary training schedule;
- the entropic pump as a learned forcing term.

The entropic clock is implemented as an intrinsic stopping criterion in Section 8: the sampler terminates when the spectrally-weighted heat-death metric drops below a threshold.

4 The 2.5-D latent

4.1 Dual representation

Let $x \in \mathbb{R}^{H\times W\times 3}$ be an image. The encoder produces two coupled outputs:

$$x \xrightarrow{E_\phi} (z, A),$$

where $z \in \mathbb{R}^{h\times w\times c}$ is the spatial latent and $A \in \mathbb{R}^{N\times F}$ is a feature field ($N = h \cdot w$).
Implementation: `vae.py` – `SpectralVAE.encode()` produces $(z, A, \mu, \mathrm{logvar})$ via a shared conv trunk, a spatial head (Conv2d $\to \mu$, logvar), and a feature head (adaptive pool $\to$ linear projection to F dimensions).

4.2 Smooth projection

The ArrowSpace prior defines a smooth subspace through the projection $\Pi_q = U_q U_q^\top$. Applying this to the feature field yields $A_\parallel = A\Pi_q$ and $A_\perp = A(I - \Pi_q)$. The parallel component lies in the smooth semantic subspace; the perpendicular captures off-manifold residual.
Implementation: `arrow_prior.py` – `ArrowSpacePrior.project_to_chart(A)` computes $AU_qU_q^\top$. The prior stores L_F, U_q, eigenvalues, and energy-dispersion as non-trainable buffers (zero `nn.Parameter`).

4.3 Spectral chart

From the projected feature field we extract chart coordinates $s = \mathrm{Pool}(A_{\|})U_q \in \mathbb{R}^q$, per-mode band energies $e_k = \frac{1}{N}\|Au_k\|_2^2$, normalised energies $\tilde{e}_k = e_k/(\sum_j e_j + \varepsilon)$, the projected energy-dispersion prior $\lambda^{\mathrm{ED}}_{k^{\mathrm{chart}}} = \sum_f \lambda_f^{\mathrm{ED}} U_{f,k}^2$, and the full conditioning vector

$$c_{\mathrm{spec}} = [\tilde{e}_1, \ldots, \tilde{e}_q,\ \lambda^{\mathrm{ED}}_{1^{\mathrm{chart}}}, \ldots, \lambda^{\mathrm{ED}}_{q^{\mathrm{chart}}},\ \nu_1, \ldots, \nu_q] \in \mathbb{R}^{3q}.$$

Implementation: `arrow_prior.py` – `chart_energy_descriptor(A)` returns c_{spec} of shape $(B, 3q)$. No MLP is applied in the current implementation; the raw $3q$ vector is used directly as conditioning. An MLP can be added without changing the interface.

5 Model architecture

5.1 Overview

ALD-SC consists of four components:

1. a frozen ArrowSpace prior $(L_F, U_q, \Lambda_q, \lambda^{\mathrm{ED}})$;
2. a spectral VAE encoder producing z and A;
3. a latent diffusion denoiser (DiT) operating only on z;
4. a topology-adaptive decoder conditioned on c_{spec}.

The full pathway is

$$x \xrightarrow{E_\phi} (z, A) \xrightarrow{\text{chart}} (z, c_{\mathrm{spec}}) \xrightarrow{\text{diffusion}} \hat{z} \xrightarrow{D_\psi(\cdot|c_{\mathrm{spec}})} \hat{x}.$$

5.2 Encoder

The encoder is a shared visual trunk with two heads. The spatial head produces $z \sim q_\phi(z \mid x)$ via mean and log-variance fields. The feature head produces $A = E_A(x) \in \mathbb{R}^{N \times F}$. The feature field is not diffused; it is projected onto the ArrowSpace subspace and summarised into c_{spec}.

5.3 Diffusion backbone

We apply diffusion only to the spatial latent:

$$q(z_t \mid z_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} z_0,\ (1 - \bar{\alpha}_t) I).$$

A DiT denoiser predicts velocity:

$$v_\theta(z_t, t, c_{\mathrm{text}}, c_{\mathrm{spec}}).$$

The spectral chart enters via adaptive layer normalisation (AdaLN): the conditioning vector c_{spec} is fused with the timestep embedding and projected to per-block scale and shift parameters. Classifier-free guidance (CFG) dropout on c_{spec} is applied during training.
Implementation: `dit.py` – `MinimalDiT` with patchify, sinusoidal time embedding, AdaLN blocks, and CFG dropout. `schedule.py` provides `CosineSchedule` and `LinearSchedule` with `add_noise`, `v_target`, `sample_batch`.

5.4 Graph-structured decoder

The decoder reconstructs pixels by propagating information along the ArrowSpace graph's smooth directions, gated by the dispersion network. This replaces the standard VAE decoder's unconstrained convolutions with a constructive decoding operator where L_F (via U_q) defines the reconstruction paths and λ^{ED} (via c_{spec}) defines the energy allocation.

At each resolution stage, the *WaveReconstructionBlock* performs:

1. Pool feature activations: $A = \text{Pool}(H^{(\ell)})$
2. Project to chart: $\hat{H} = AU_q$ (decode along smooth directions)
3. Gate by dispersion: $g = \sigma(W\, c_{\text{spec}})$
4. Lift back: $A' = (\hat{H} \odot g)U_q^\top$ (reconstruct in feature space)
5. Residual update: $H^{(\ell+1)} = H^{(\ell)} + \text{Conv}(\text{norm}(H^{(\ell)} + \Delta))$

where Δ is the feature-space reconstruction projected back to channel dimension. This is the graph-theoretic analogue of the VAE reparameterization trick: information flows along the graph's eigenvectors rather than through unconstrained learned weights.

Clock-gated tempo. The *ClockGatedGraphDecoder* modulates the strength of each wave block by the spectral schedule's $\bar{\alpha}_k(t)$: early in the denoising process (high noise), gates are weak (minimal graph-structured reconstruction); late (low noise), gates are strong (full graph-structured reconstruction). This implements the Barontini principle within the decoder: the graph geometry governs *when* reconstruction effort is allocated.

Implementation: `graph_decoder.py` – `GraphDecoder` uses 3 `WaveReconstructionBlock` instances at decreasing channel resolution (4ch → 2ch → ch). `ClockGatedGraphDecoder` adds the `SpectralSchedule` for tempo modulation. The `WaveReconstructionBlock` stores U_q as a frozen buffer (zero trainable parameters in the graph structure itself). The simplified single-gate decoder in `vae.py` remains for backward compatibility.

6 Heat-kernel prior

The spectral chart admits a natural diffusion-geometric prior. Let $s = (s_1, \ldots, s_q)^\top$ denote the chart coordinates. We define

$$p_t(s) = \mathcal{N}\big(0,\ \text{diag}\big[w_k e^{-2t\nu_k}\big]_{k=1}^{q}\big)\,,$$

where $w_k = \frac{\exp(-\rho\lambda^{\text{ED}}_{k\text{chart}})}{\sum_j \exp(-\rho\lambda^{\text{ED}}_{j\text{chart}})}$. Low eigenvalue modes persist under diffusion; high eigenvalue modes decay rapidly. The energy-dispersion weights bias the chart toward regions of spectral space occupied by corpus data.

Definition 1 (2.5-D manifold prior)**.** *The pair (z, s) defines a 2.5-D manifold prior if z carries local spatial degrees of freedom and s is drawn from a heat-kernel distribution on the ArrowSpace spectral chart.*

7 Objective

The training objective is

$$\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} + \lambda_{\text{chart}}\mathcal{L}_{\text{chart}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}} + \lambda_{\text{kl}}\mathcal{L}_{\text{kl}}.$$

7.1 Latent diffusion loss

Using velocity prediction,

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{z_0,\epsilon,t}\left[\|v - v_\theta(z_t, t, c_{\text{spec}})\|_2^2\right].$$

7.2 Reconstruction and VAE regularisation

$$\mathcal{L}_{\text{rec}} = \|x - \hat{x}\|_1 + \lambda_{\text{perc}}\mathcal{L}_{\text{perc}}(x, \hat{x}), \qquad \mathcal{L}_{\text{kl}} = D_{\text{KL}}\big(q_\phi(z \mid x)\,\|\,p(z)\big).$$

7.3 Spectral consistency

Re-encoding $\hat{x}$ produces $\hat{A}$; we require the spectral band allocation to match:

$$\mathcal{L}_{\text{chart}} = \left\|\tilde{e}(A) - \tilde{e}(\hat{A})\right\|_2^2.$$

7.4 Off-manifold penalty

$$\mathcal{L}_{\text{smooth}} = \frac{\|A(I - U_q U_q^\top)\|_F^2}{\|A\|_F^2 + \varepsilon}.$$

Implementation: `losses.py` – `ALDSCLoss` computes all five terms. The VAE KL is included in the current implementation (`vae.kl_loss()` extracts μ and logvar from the last forward pass).

8 Entropic time from the ArrowSpace spectrum

Implementation note. The Barontini-inspired entropic clock is an active part of the ALD-SC inference pipeline. The heat-death stopping criterion (Section 8.5) is wired into the DDIM and Euler samplers, terminating sampling when $\sum_k \nu_k \cdot \bar{\alpha}_k(t) < \varepsilon$.

The heat-kernel prior admits a natural reading in terms of entropic time. The ArrowSpace Laplacian eigenvalues play the role of Barontini's barrier-controlled entropy exchange rates, yielding a per-mode entropic clock that requires no additional learned parameters and no auxiliary model. This connects the frozen spectral prior directly to information-theoretic advances in diffusion scheduling [10].

8.1 Entropy rate of the heat-kernel chart

The per-mode variance under the heat-kernel prior is $\sigma_k^2(t) = w_k e^{-2t\nu_k}$. The Shannon entropy of a Gaussian with this variance is $S_k(t) = \frac{1}{2}\log(2\pi e\, w_k) - t\nu_k$, so the entropy rate is

$$\boxed{\frac{dS_k}{dt} = -\nu_k.}$$

Each Laplacian eigenvalue ν_k *is* the entropy exchange rate for mode k.

8.2 Correspondence to Barontini's two-sector clock

Barontini partitions a closed Bose–Einstein condensate into a bright (observed) and dark (unobserved) sector, separated by a tunable potential barrier of height V. Entropic time is constructed from the absolute entropy exchange between sectors. Table 1 maps this onto the ALD-SC spectral chart.

Table 1: Correspondence between Barontini's BEC experiment and the ALD-SC spectral chart.

Barontini experiment	**ALD-SC spectral chart**
Bright sector (observed)	Active modes: low ν_k, high variance
Dark sector (unobserved)	Inactive modes: high ν_k, collapsed variance
Barrier height V	Laplacian eigenvalue ν_k (per-mode barrier)
Entropy exchange rate	$\lvert dS_k/dt\rvert = \nu_k$
Entropic time τ	Per-mode $\tau_k(t) = \nu_k \cdot t$
Big bang / big crunch	Mode activation / collapse during denoising
Heat death	All modes at equilibrium variance
Barrier-height sweep	Varying q (number of retained modes)

8.3 Per-mode entropic schedule and stopping criterion

Instead of a single global schedule $\bar{\alpha}(t)$, each mode receives its own entropic schedule driven by its internal clock:

$$\tau_k(t) = \nu_k \cdot t, \qquad \bar{\alpha}_k(\tau_k) = \cos^2\left(\frac{\pi}{2} \cdot \frac{\tau_k}{\tau_{k,\max}}\right),$$

where $\tau_{k,\max} = \nu_k \cdot T$ for a reference horizon T.

Remark 2. ***Mathematical observation.*** *Since ν_k cancels in the ratio $\tau_k/\tau_{k,\max} = t/T$, all modes have identical $\bar{\alpha}_k$ in external time. The per-mode differentiation arises through the* heat-death metric $\sum_k \nu_k \cdot \mathrm{mmse}_k(t)$*, which is weighted by ν_k: high-ν_k modes dominate the stopping criterion. All modes formally reach equilibrium at the same external time T, but high-ν_k modes accumulate entropic time τ_k faster.*

Implementation: `spectral_schedule.py` – `SpectralSchedule` implements $\tau_k(t)$, $\bar{\alpha}_k(t)$, the entropy rate $dS_k/dt = -\nu_k$, and the heat-death stopping criterion $\sum_k \nu_k \cdot \bar{\alpha}_k(t) < \varepsilon$. The samplers in `sampling.py` accept an optional `spectral_schedule` parameter; when provided, sampling stops early at heat death instead of at a fixed step count. All components are frozen (zero `nn.Parameter`). 21 unit tests verify shapes, monotonicity, boundary cases, the ν_k cancellation, and the spectral stopping criterion.

8.4 Connection to information-theoretic schedulers

Stancevic et al. [10] prove that the conditional entropy $\mathbf{H}[\mathbf{x}_0 \mid \mathbf{x}_t]$ is a schedule-invariant time reparameterisation for diffusion models. The entropy rate decomposes over modes when the chart covariance is diagonal:

$$\dot{\mathbf{H}}[s_0 \mid s_t] = \sum_{k=1}^{q} \nu_k \cdot \mathrm{mmse}_k(t).$$

The ALD-SC contribution is that the ArrowSpace spectrum provides the decomposition basis a priori: the ν_k are frozen graph eigenvalues.

Table 2 contrasts ALD-SC with prior approaches.

8.5 Heat death as a stopping criterion

In ALD-SC, heat death corresponds to all modes reaching equilibrium variance:

$$\text{heat death} \iff \sum_{k=1}^{q} \nu_k \cdot \mathrm{mmse}_k(t) < \varepsilon.$$

This provides an intrinsic, data-dependent stopping criterion for sampling.

Table 2: Comparison of noise-scheduling approaches.

Feature	**Stancevic et al.**	**Spectrally-Guided (Esteves)**	**ALD-SC (this work)**
Time variable	Cond. entropy	RAPSD profile	Per-mode τ_k
Spectral basis	Fourier (pixel)	Fourier (pixel)	ArrowSpace (semantic)
Per-mode schedule	Spectral variant	No	τ_k (stopping only)
Geometry source	Image frequency	Image spectrum	Feature-space graph
Online adaptation	No (post-train)	No	No (static ν_k)
Barrier analogue	None	None	q (retained modes)

9 Minimal viable experiment

We implement the full ALD-SC pipeline and validate it on a controlled toy experiment. The goal is not to claim state-of-the-art image quality but to demonstrate that the spectral chart pipeline is functional and that c_{spec} measurably affects generation.

9.1 Setup

- **Prior:** $F = 32$, $q = 8$, $k = 4$ (kNN over feature columns). Built from 64 synthetic embeddings.
- **VAE:** 3-channel input, 4-channel latent ($32 \times 32 \to 8 \times 8$ latent), 32 base channels.
- **DiT:** patch size 2, dim 64, depth 4, 4 heads, c_{spec} dim $= 3q = 24$, CFG dropout 0.1.
- **Schedule:** Cosine, 1000 steps, v-prediction.
- **Training:** 20 epochs VAE, 20 epochs diffusion, Adam lr$= 10^{-3}$, batch size 8, seed 3407.
- **Data:** 32 synthetic 32×32 noise images (ToyImageDataset).

9.2 Results

- **Image generation:** `scripts/sample.py` generates and saves a PNG end-to-end.
- **Conditioning sensitivity:** swapping c_{spec} between images produces measurably different DiT velocity predictions (verified by unit test after perturbing zero-initialised AdaLN weights).
- **CFG dropout:** at dropout probability 1.0, the model falls back to the unconditional path; at 0.0, c_{spec} is always used.
- **Frozen prior:** verified that the prior has zero trainable parameters and that VAE parameters stay frozen during diffusion training.
- **Graph-structured decoding:** the `GraphDecoder` uses `WaveReconstructionBlock` instances that propagate information along U_q directions, gated by c_{spec}. The `ClockGatedGraphDecoder` modulates gate strength by the spectral schedule (verified: different diffusion times produce different outputs).
- **Spectral stopping:** the per-mode $\tau_k(t)$, $\bar{\alpha}_k$, entropy rate, and heat-death metric are computed on frozen ν_k. The DDIM sampler terminates early when the heat-death criterion is met (verified by unit test: high-ε stops before max steps; same seed is deterministic).

9.3 Limitations

- Experiments use synthetic noise data, not real images. Real data experiments (CIFAR-10 with DINO/SigLIP embeddings) are the next step.
- The graph decoder uses a single graph-filter step per block, not the full second-order wave recurrence from ESDM.
- The entropic clock provides an intrinsic stopping criterion but not an active training schedule; the cosine schedule is still used during training.
- No FID or quantitative image-quality metrics are reported.

9.4 Metrics (for future real-data experiments)

Beyond standard image-quality metrics (PSNR, SSIM, LPIPS, FID), evaluation should include spectral diagnostics:

- chart-energy error: $\|\tilde{e}(x) - \tilde{e}(\hat{x})\|^2$;
- off-manifold energy ratio;
- chart interpolation smoothness;
- global semantic coherence under low latent capacity.

10 Connection to internal time

Although ALD-SC does not use the full entropic clock as its primary training schedule, the original motivation remains important. The conceptual bridge is that geometry and dynamics should be derived from internal state rather than imposed entirely from outside. In ESDM this took the form of an entropy-derived temporal variable. In ALD-SC it takes the form of a feature-space geometric prior that governs which semantic directions are valid and how their energy is distributed. The spectral schedule (Section 8.3) provides the active stopping criterion, making the Barontini principle operational within the minimal architecture.

11 Conclusion

We presented ALD-SC, a minimal spectrally conditioned latent diffusion architecture grounded in ArrowSpace feature geometry. The model combines a standard spatial latent with a frozen spectral prior and a decoder conditioned on a compact ArrowSpace chart. This realises the central 2.5-D claim: local image detail lives in a spatial latent plane, while global semantic coherence is governed by a lower-dimensional graph manifold.

The full ALD-SC pipeline — prior construction, spectral VAE, v-prediction diffusion, and DDIM/Euler sampling — is implemented and validated by 67 unit tests. The entropic clock is included as an active intrinsic stopping criterion in the inference pipeline, making the Barontini principle operational within the minimal architecture. The upgrade path to the full ESDM remains open through the entropic training schedule.

The original entropic formulation remains an important theoretical horizon. The present architecture offers a cleaner and more tractable route to testing the key scientific idea: that generative models should reconstruct and sample not only in compressed latent coordinates but on a semantic manifold defined by the graph Laplacian structure of the feature space itself.

Acknowledgments

The authors acknowledge the use of AI-assisted tools during the preparation of this paper, including Perplexity for literature discovery, OpenAI GPT models for drafting and result analysis, NVIDIA Nemotron for exploratory formal verification analysis, Claude Sonnet 4.6 for coding support and implementation checks, and GLM-5.2 by Z.AI for code generation and experimental iteration [12, 13, 14, 15, 16]. All technical claims, interpretations, experiments, and final editorial decisions remain the sole responsibility of the authors.

A Reproducibility: code-to-paper mapping

Table 3: Mapping from paper sections to implementation files.

Paper section	File	Status
§4: 2.5-D latent	`vae.py`, `arrow_prior.py`	Implemented
§4.2: projection	`arrow_prior.py`	Implemented
§4.3: c_{spec}	`arrow_prior.py`	Implemented
§5.3: DiT + AdaLN	`dit.py`	Implemented
§5.3: schedules	`schedule.py`	Implemented
§5.4: graph decoder	`graph_decoder.py`	Implemented
§7: losses	`losses.py`	Implemented
§6: heat-kernel prior	—	Theoretical
§8: entropic clock	`spectral_schedule.py`, `sampling.py`	Implemented
§8.3: per-mode schedule	`spectral_schedule.py`	Implemented
§8.5: heat death	`spectral_schedule.py`, `sampling.py`	Implemented
§9: image generation	`scripts/sample.py`	Implemented

All code is at https://github.com/tuned-org-uk/arrowspace-latent-diffusion. Run tests: `uv run pytest tests/ -v` (67 tests, CPU). Generate an image: `uv run python scripts/sample.py`.

References

[1] L. Moriondo, Vibrational Deduction Transformer (VDT) Leveraging Spectral methods, Theory of Sound and Associative Memory. Zenodo. https://doi.org/10.5281/zenodo.20816835.

[2] G. Barontini, Testing the problem of time with cold atoms, *Physical Review Research*, 8:L022047, 2026. doi: 10.1103/1h9j-df4k.

[3] A. Connes and C. Rovelli, Von Neumann Algebra Automorphisms and Time-Thermodynamics Relation in General Covariant Quantum Theories, *Classical and Quantum Gravity*, 11:2899–2918, 1994. arXiv:gr-qc/9406019.

[4] L. Moriondo, ArrowSpace: introducing Spectral Indexing for vector search, *Journal of Open Source Software*, 10(113):9002, 2025. doi: 10.21105/joss.09002.

[5] L. Moriondo and I. Azizi, From Embedding Geometry to Spectral Search: Energy Dispersion Networks for Vector Retrieval, arXiv:2606.21535 [cs.IR], 2026. https://arxiv.org/abs/2606.21535.

[6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, High-Resolution Image Synthesis with Latent Diffusion Models, in *CVPR*, 2022. arXiv:2112.10752.

[7] A. Nichol and P. Dhariwal, Improved Denoising Diffusion Probabilistic Models, in *ICML*, 2021. arXiv:2102.09672.

[8] L. Moriondo, entropic-semantic-diffusion, Github

[9] L. Moriondo, arrowspace-latent-diffusion, Github

[10] D. Stancevic, F. Handke, and L. Ambrogioni, Entropic Time Schedulers for Generative Diffusion Models, arXiv:2504.13612 [cs.LG], 2025. arXiv:2504.13612.

[11] C. Esteves and A. Makadia, Spectrally-Guided Diffusion Noise Schedules, in *ICML*, 2026. arXiv:2603.19222.

[12] Perplexity AI, Perplexity: AI-powered answer engine, https://www.perplexity.ai, 2026.

[13] OpenAI, GPT-5: Generative Pre-trained Transformer 5, https://openai.com, 2026.

[14] NVIDIA, Nemotron: Large Language Model for Formal Verification, https://www.nvidia.com, 2026.

[15] Anthropic, Claude Sonnet 4.6, https://www.anthropic.com, 2026.

[16] Z.AI, GLM-5.2: General Language Model 5.2, https://z.ai, 2026.